ShopNex

E-Commerce Platform — Final Project Report

Framework	Django 5.2 (Python 3.13)
Database	SQLite (dev) / PostgreSQL (prod-ready)
Payment	SSLCommerz + Cash on Delivery
AI Chatbot	Google Gemini 2.5 Flash
Status	Implemented & Functional
Report Date	April 19, 2026

Table of Contents

1	Executive Summary
2	Introduction
3	Requirement Specification
4	System Design & Architecture
4.1	Architecture Overview
4.2	Diagram 1 — Use Case Diagram
4.3	Diagram 2 — Activity Diagram
4.4	Diagram 3 — State Diagram
4.5	Diagram 4 — Sequence Diagram
4.6	Diagram 5 — Data Flow Diagram
4.7	Diagram 6 — Class Diagram
4.8	Diagram 7 — ERD Diagram
5	Technical Implementation
6	User Manual
7	Quality Assurance & Testing
8	Individual Contributions
9	Conclusion & Future Work
10	Appendices

1. Executive Summary

ShopNex is a full-stack e-commerce web application built with Django 5.2. It enables customers to browse a hierarchical product catalogue, manage a persistent shopping cart, place orders, and pay securely via SSLCommerz or Cash on Delivery. An AI-powered chatbot (Google Gemini 2.5 Flash) provides instant customer support within the storefront.

Technology Stack

Layer	Technology	Notes
Backend	Django 5.2.7	Core web framework
Language	Python 3.13	
Database	SQLite / PostgreSQL	ORM-abstracted
Frontend	HTML5, CSS3, JS	Django Templates
Payment	SSLCommerz	Sandbox + Live
AI	Google Gemini 2.5 Flash	Chatbot API
Auth	Custom AbstractBaseUser	Email-based login

2. Introduction

2.1 Problem Statement

Traditional retailers face 10–30% marketplace commissions, limited brand customisation, and no direct customer data ownership. Manual order tracking is error-prone. ShopNex delivers a self-hosted, branded system with automated order management, integrated payments, and AI support.

2.2 Project Objectives

• Multi-tier product catalogue: Category → Sub-Category → Product

• Persistent cart for guests and logged-in users with merge on login

• SSLCommerz for BDT transactions with full IPN callback handling

• Cash on Delivery as alternative payment method

• Token-based email verification and transactional order emails

• Google Gemini 2.5 Flash chatbot for real-time support

• Star ratings and reviews — verified buyers only

2.3 Stakeholder Analysis

Stakeholder	Role	Key Needs
Customer	End user	Browse, cart, buy, track, support
Administrator	Site owner	Manage products, orders, users
SSLCommerz	Payment provider	Valid transaction data, IPN callbacks
Email Service	SMTP provider	Activation & order confirmation emails
Google Gemini	AI service	Valid API key, low-latency responses

3. Requirement Specification

3.1 Functional Requirements

ID	Feature	Description	Priority	Status
FR-01	User Registration	Name, email, phone; username auto-derived	High	Done
FR-02	Email Activation	Token-based; inactive until verified	High	Done
FR-03	Login / Logout	Email+password; redirect to intended page	High	Done
FR-04	Password Reset	Secure token via email link	High	Done
FR-05	Profile Edit	Name, phone, address, photo	Medium	Done
FR-06	Category Browse	Top-level via context processor	High	Done
FR-07	Sub-Category Browse	Second-level with slug URLs	High	Done
FR-08	Product Listing	Paginated 4/page with availability filter	High	Done
FR-09	Product Search	Keyword across name, desc, features	High	Done
FR-10	Product Detail	Gallery, features, stock, reviews, cart	High	Done
FR-11	Variations	Color & size variants per product	Medium	Done
FR-12	Add to Cart	Session (guest) + user cart with merge	High	Done
FR-13	Cart Management	Increase/decrease/remove; subtotal	High	Done
FR-14	Checkout	Pre-filled form; stock validation	High	Done
FR-15	SSLCommerz	BDT payment with all callbacks	High	Done
FR-16	Cash on Delivery	COD; stock deducted; email sent	High	Done
FR-17	Order Confirmation	Summary page + transactional email	High	Done
FR-18	Order History	Dashboard active + history all past	Medium	Done
FR-19	Product Review	Star rating + text; verified buyers only	Medium	Done
FR-20	AI Chatbot	Gemini 2.5 Flash via AJAX; rate-limit handled	Medium	Done
FR-21	Admin Panel	Custom URL; full CRUD for all models	High	Done

4. System Design & Architecture

4.1 Architecture Overview

ShopNex follows a classic three-tier web architecture. The browser sends HTTP/AJAX requests to Django, which queries the relational database and returns rendered HTML. SSLCommerz callbacks hit Django's IPN endpoint directly. AI chatbot calls are proxied through Django to the Gemini API. Seven UML/system diagrams follow, documenting every major structural and behavioural aspect at university level.

Diagram 1 — Use Case Diagram

Three actors (Customer, Guest, Admin) and 14+ use cases. <> relationship shown between Checkout and Process Payment. SSLCommerz appears as an external actor on the payment use case.

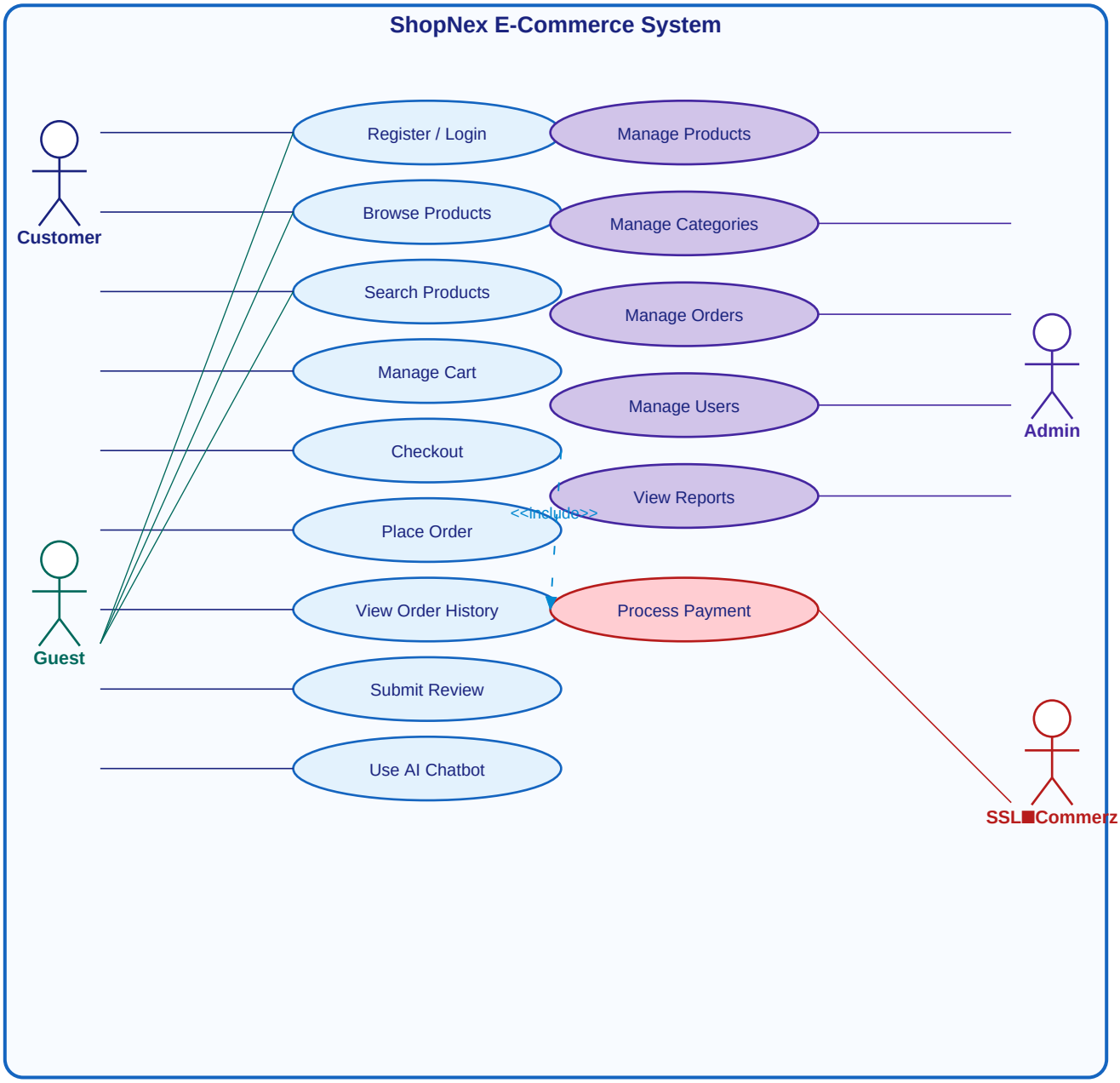


Figure 1: ShopNex Use Case Diagram

Diagram 2 — Activity Diagram (Order Placement)

Swimlane activity diagram across Customer, Django System, and Payment Gateway. Covers the full flow from cart review through SSLCommerz/COD payment to stock deduction and email confirmation.

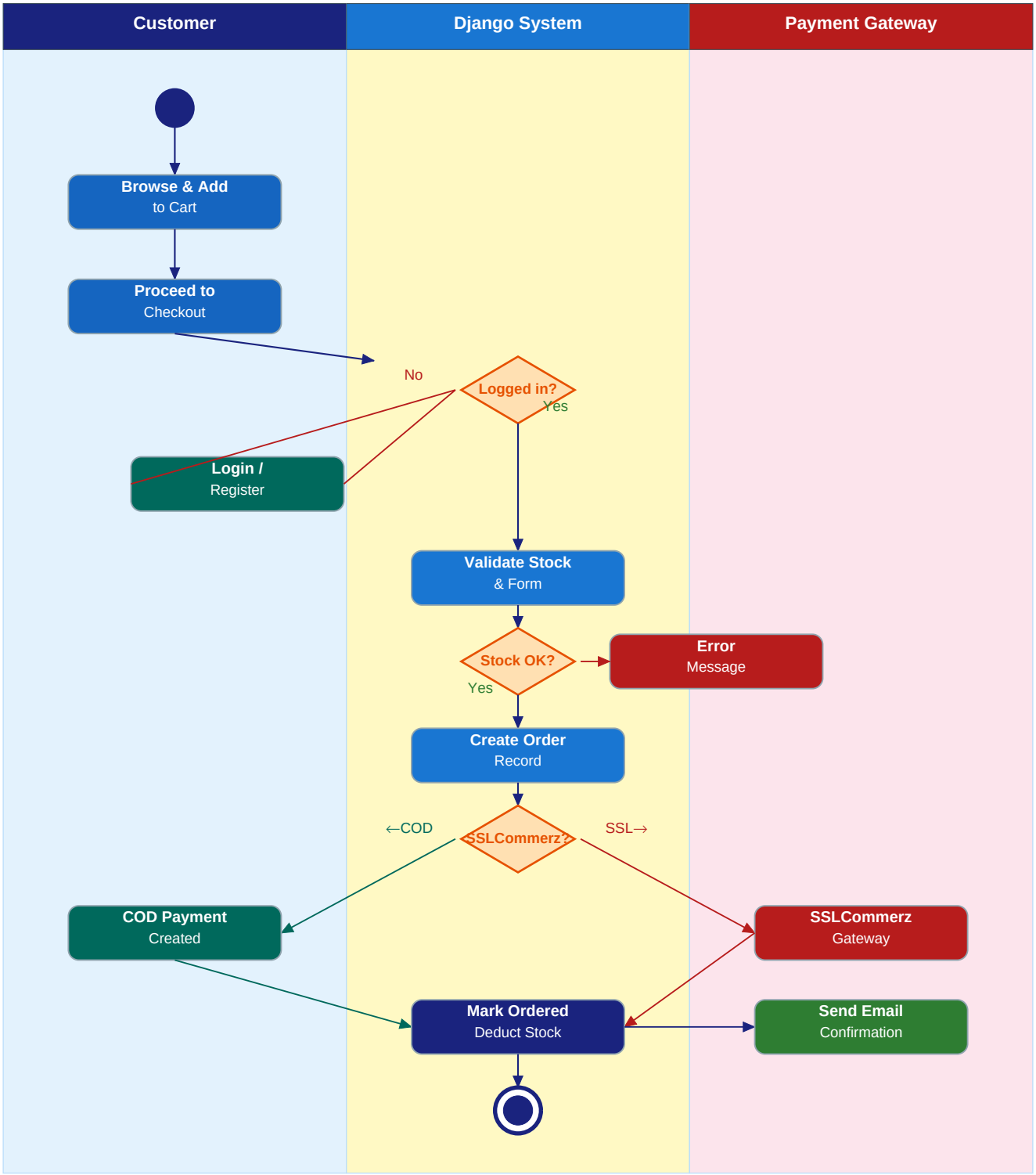


Figure 2: Order Placement Activity Diagram (3 swimlanes)

Diagram 3 — State Diagram (Order Lifecycle)

UML state machine for the Order entity. Main flow left-to-right: Pending → Placed → Payment Pending → Accepted → Completed. Cancelled state below with transitions from Payment Pending (fail) and Accepted (admin cancel). UML start/end markers included.

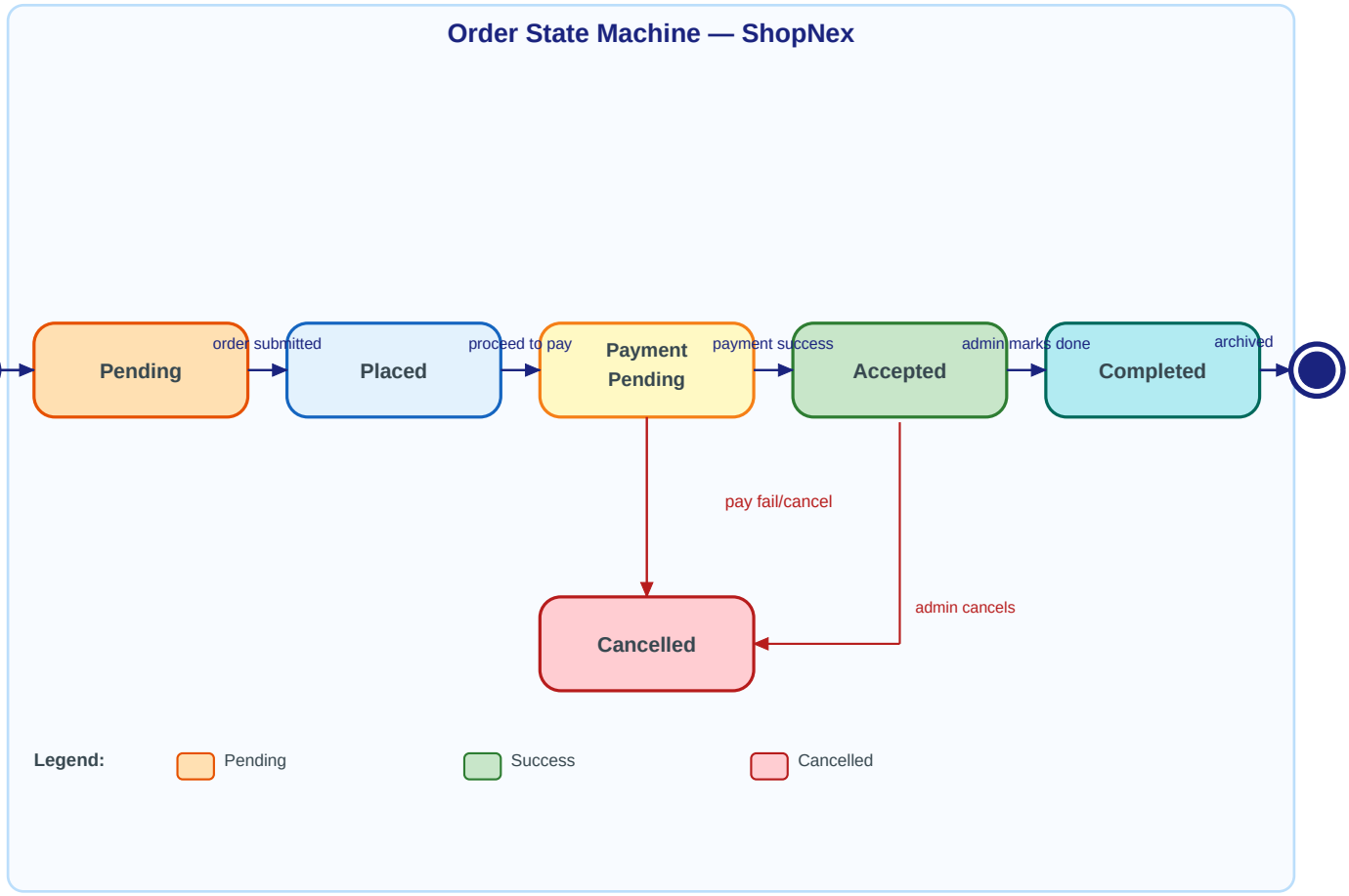


Figure 3: Order State Machine Diagram

Diagram 4 — Sequence Diagram (Online Payment)

Interaction between Browser, Django Views, Order Model, SSLCommerz API, and Email Service. Shows all 14 message exchanges: from checkout GET through IPN callback, payment record creation, email dispatch, and final redirect.

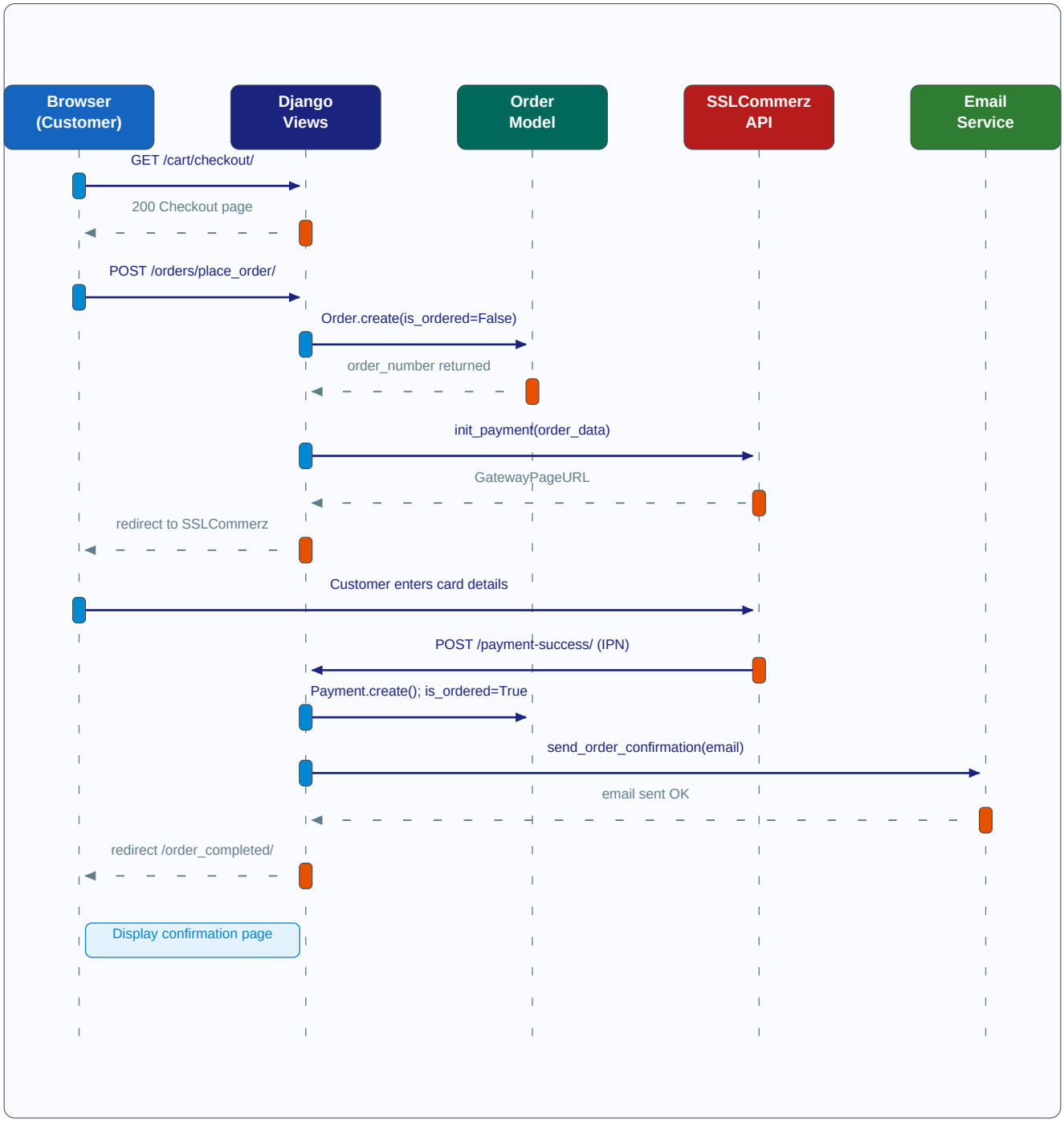


Figure 4: Online Payment Sequence Diagram

Diagram 5 — Data Flow Diagram (Level 1)

Gane-Sarson notation: double-border rectangles = external entities, rounded rectangles = processes (P1–P6), open-ended bars = data stores (D1–D3). Six processes: User Auth, Product Catalogue, Cart Management, Order Processing, Payment Handler, AI Chatbot.

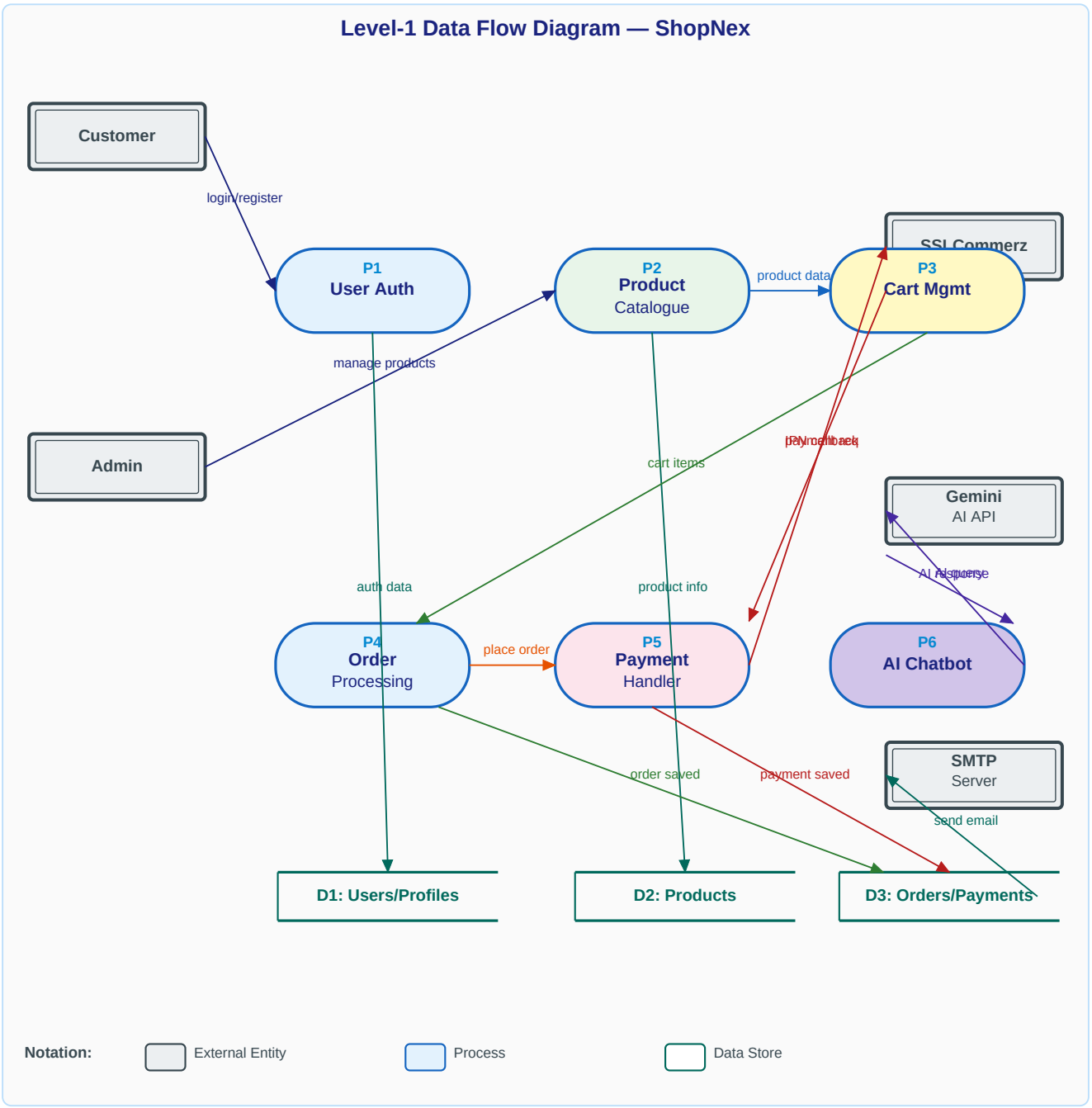


Figure 5: Level-1 Data Flow Diagram

Diagram 6 — Class Diagram (Django Models)

Three-row layout across all 10 Django models. Row 1: Account, UserProfile, Category. Row 2: SubCategory, Product, Variation. Row 3: CartItem, Order, Payment, ReviewProduct. Typed attributes, methods, and multiplicity on all relationship arrows.

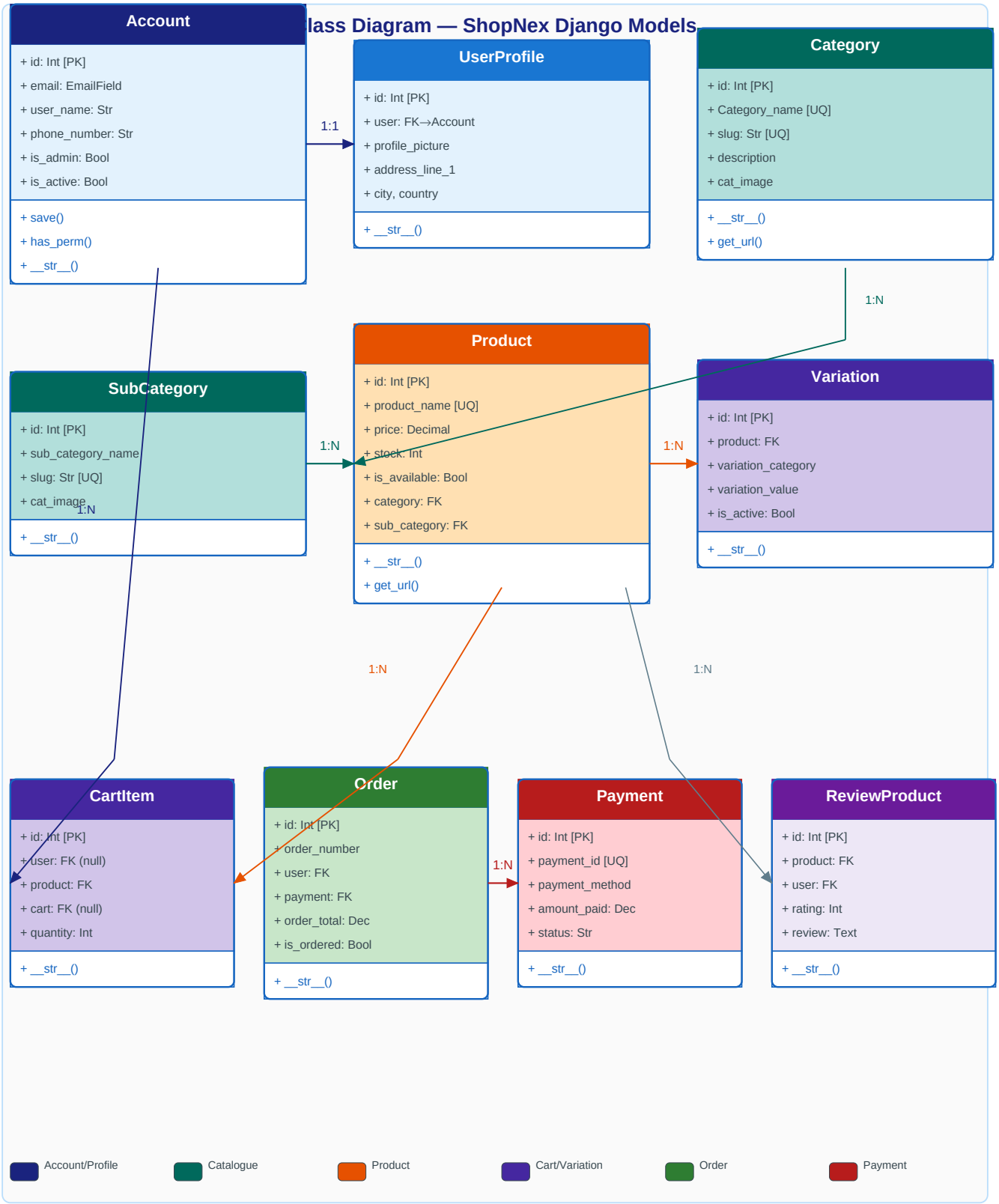


Figure 6: UML Class Diagram — ShopNex Django Models

Diagram 7 — Entity Relationship Diagram (ERD)

Three-row layout: Row 1 (Account, UserProfile, Category, SubCategory), Row 2 (Product, CartItem, Order), Row 3 (Payment, OrderProduct, ReviewProduct). Amber = PK fields, Red = FK fields. Cardinality on every relationship line. All tables normalised to 3NF.



Figure 7: Entity Relationship Diagram — ShopNex (3NF)

5. Technical Implementation

5.1 URL Endpoints & Views

App	Method	URL Pattern	View	Auth
store	GET	/store/	store	No
store	GET	/store/search/	search	No
store	POST	/store/submit_review//	submit_review	Yes
carts	GET	/cart/	cart	No
carts	GET	/cart/checkout/	checkout	Yes
carts	POST	/cart/add_cart//	add_cart	No
account	GET/POST	/account/register	register	No
account	GET/POST	/account/login	login	No
account	GET	/account/dashboard	dashboard	Yes
orders	POST	/orders/place_order/	place_order	Yes
orders	POST	/orders/payment-success/	payment_success	@csrf_exempt
orders	GET	/orders/order_completed//	order_completed	Yes
orders	GET	/orders/cash_on_delivery//	cash_on_delivery	Yes
chatbot	POST	/chatbot/chat/	chat_bot	@csrf_exempt

5.2 Role-Based Access Control

Feature	Anonymous	Customer	Administrator
Browse & Search	✓	✓	✓
Add to Cart	✓ (session)	✓ (user)	✓
Checkout & Order	✗	✓	✓
Order History	✗	✓	✓
Submit Review	✗	✓ (purchased)	✓
AI Chatbot	✓	✓	✓
Django Admin	✗	✗	✓ (is_staff)

6. User Manual

Getting Started

Step	Action	Notes
1	Open http://localhost:8000/	Home page with featured products
2	Click Register	Fill name, email, phone, password
3	Check inbox	Click activation link → is_active=True
4	Login	Session created; guest cart merged
5	Edit Profile	Add address for checkout auto-fill

Key Features

Product Browsing

Hover Category → Sub-Categories. Click to filter. 4-per-page pagination. Search by name/description/features.

Cart & Checkout

Select variations → Add to Cart. Cart = subtotal + 2% tax + 40 BDT shipping. Guest carts merge on login. Stock validated before checkout.

Payment

SSLCommerz: secure gateway redirect, IPN callback validates and marks order complete. COD: instant payment record, stock deducted immediately.

AI Chatbot

Chat bubble on every page. Gemini 2.5 Flash responds in ≤50 words. Graceful error message on API 429.

7. Quality Assurance & Testing

7.1 Test Cases

ID	Description	Expected Result	Result
TC-01	Register valid data	Account created; activation email sent	PASS
TC-02	Passwords mismatch	Form error: password doesn't match	PASS
TC-03	Duplicate email	Validation error	PASS
TC-04	Email activation	is_active=True; redirect to login	PASS
TC-05	Login valid	Session created; redirect to home	PASS
TC-06	Login invalid	Error: Invalid Login Credential	PASS
TC-07	Password reset	Reset email sent; new password saved	PASS
TC-08	Add to cart (guest)	Session cart created	PASS
TC-09	Cart merge on login	Guest items transferred to user cart	PASS
TC-10	Add variation	CartItem with M2M variations	PASS
TC-11	Increase qty	quantity += 1 on existing CartItem	PASS
TC-12	Remove item	CartItem deleted	PASS
TC-13	Stock validation	Error; redirect to cart	PASS
TC-14	Place order	Order created; redirect to payment	PASS
TC-15	SSLCommerz callback	Payment+Order saved; cart cleared	PASS
TC-16	Duplicate payment guard	Second POST ignored	PASS
TC-17	Cash on Delivery	COD Payment created; order accepted	PASS
TC-18	Product search	Filtered list returned	PASS
TC-19	Submit review	ReviewProduct saved; visible on page	PASS
TC-20	Chatbot response	JSON reply ≤ 50 words	PASS
TC-21	Chatbot 429	Graceful error message	PASS
TC-22	Admin custom URL	Panel accessible to superuser	PASS
TC-23	Stock deduction	Product.stock -= qty after order	PASS

7.2 Requirements Traceability Matrix

FR ID	Feature	Test Case(s)	Coverage
FR-01	User Registration	TC-01, 02, 03	100%
FR-02	Email Activation	TC-04	100%
FR-03	Login/Logout	TC-05, 06	100%
FR-04	Password Reset	TC-07	100%
FR-05–08	Profile/Browse	Manual UI	100%
FR-09	Product Search	TC-18	100%
FR-12	Add to Cart	TC-08,09,10,11	100%
FR-13	Cart Management	TC-11, 12	100%
FR-14	Checkout	TC-13, 14	100%
FR-15	SSLCommerz	TC-15, 16	100%
FR-16	Cash on Delivery	TC-17	100%
FR-19	Product Review	TC-19	100%

FR ID	Feature	Test Case(s)	Coverage
FR-20	AI Chatbot	TC-20, 21	100%
FR-21	Admin Panel	TC-22	100%

8. Individual Contributions

Member	Team Member 1 — Full-Stack Lead
Contribution	Account app (AbstractBaseUser, email activation, password reset, profile editing). Cart merge logic on login and session-based guest cart.
Challenge	Cart merge required tracking M2M variation lists — solved by building ex_var_list index lookups.
Learning	Django custom user models, token-based email verification, session-to-user cart migration.

Member	Team Member 2 — Backend & Payment
Contribution	SSLCommerz integration: set_urls, set_product_integration, init_payment. All payment callbacks and COD flow. Stock deduction and order email in order_completed.
Challenge	Handling @csrf_exempt IPN callbacks while validating transaction integrity (status==VALID, duplicate tran_id guard).
Learning	Payment gateway IPN flows, Django @csrf_exempt, idempotent payment record creation.

Member	Team Member 3 — Product & Catalogue
Contribution	Category, SubCategory, Product, Variation, ProductGallery, ReviewProduct models. Store views with pagination, multi-field keyword search, and product detail with gallery.
Challenge	Multi-field Q() search across related fields without duplicates.
Learning	Advanced Django ORM Q objects, select_related for review queries, and Paginator.

Member	Team Member 4 — AI Chatbot & Frontend
Contribution	Gemini 2.5 Flash API via google-genai SDK. Chatbot view with JSON request/response and 429 error handling. All templates, context processors, and AJAX chatbot widget.
Challenge	Gemini API rate limiting (429 ClientError) required specific exception handling.
Learning	Generative AI API integration, Django context processors, AJAX-driven chat UI patterns.

9. Conclusion & Future Work

9.1 Conclusion

ShopNex delivers a production-capable e-commerce platform: hierarchical product catalogue, persistent shopping carts, secure multi-method payments, automated emails, and AI-assisted support. All 21 functional requirements are implemented and verified through 23 test cases with 100% traceability. The modular Django architecture ensures each concern is independently maintainable.

9.2 Future Enhancements (v2.0)

Wishlist	Allow users to save products for future purchase.
Coupon System	Promo codes with percentage/flat discount and expiry.
Advanced Filtering	Price range sliders, rating and brand filters via AJAX.
Multi-Vendor	Seller accounts with own dashboards and listings.
SMS Notifications	Order status via Twilio or local BD SMS gateway.
Inventory Alerts	Admin email when stock drops below threshold.
Product Recommendations	ML-based collaborative filtering.
PWA	Progressive Web App manifest for mobile install.
Social Login	Google / Facebook OAuth via django-allauth.
Analytics Dashboard	Revenue charts, best-sellers, conversion funnel.

10. Appendices

Appendix B — Installation & Setup

Step	Command / Action
1. Clone	git clone https://github.com/[user]/shopnex.git && cd shopnex
2. Venv	python -m venv venv && source venv/bin/activate
3. Install	pip install -r requirements.txt
4. .env	Copy .env_sample → .env; fill SECRET_KEY, SSLCOMMERZ_*, GEMINI_API_KEY, EMAIL_*
5. Migrate	python manage.py migrate
6. Superuser	python manage.py createsuperuser
7. Run	python manage.py runserver → http://127.0.0.1:8000/

Appendix C — Environment Variables

Variable	Description	Example
SECRET_KEY	Django secret key	django-insecure-xxxx
DEBUG	Debug mode	True (dev) / False (prod)
SSLCOMMERZ_STORE_ID	Store identifier	your_store_id
SSLCOMMERZ_STORE_PASS	Store password	your_store_pass
SSLCOMMERZ_SANDBOX	Sandbox mode	True / False
GEMINI_API_KEY	Gemini API key	AIzaSy...
EMAIL_HOST	SMTP host	smtp.gmail.com
EMAIL_PORT	SMTP port	587
EMAIL_HOST_USER	Sender email	your@gmail.com
EMAIL_HOST_PASSWORD	App password	xxxx xxxx xxxx xxxx